Lab 1: Create a GET Request Form

Objective: Understand GET requests and retrieve form data.

Problem Statement:

Create a search form that accepts a student's name and displays the submitted name on another page.

Requirements:

- Create a GET route /student/search.
- Create a Blade form with a name input.
- Submit the form using GET.
- Retrieve the submitted name using the Request object.
- Display the result.

Expected Output:

Student Search

Sample interface

Enter Student Name

Rahul Das

Search Student

Search Result: Rahul Das

Lab 2: Create a POST Request Form

Objective: Understand POST requests and request input handling.

Problem Statement:

Create an employee registration form that accepts:

- Employee name
- Email address
- Phone number
- Department

Submit the form using POST and display all submitted information.

Requirements:

- Create GET and POST routes.
- Create a Blade form.
- Use the Request object.

- Retrieve values using $request->input().
- Display submitted information.

Expected Output:

Employee Registration Successful!

Name: Rahul Das

Email: rahul@gmail.com

Phone: 9876543210

Department: IT

Lab 3: Implement CSRF Protection

Objective: Understand and implement CSRF protection.

Problem Statement:

Create a contact form using the POST method and implement Laravel CSRF protection.

Requirements:

1. Create a contact form.
2. Add @csrf inside the form.
3. Submit the form using POST.
4. Display the submitted information.
5. Remove @csrf temporarily and observe the result.

Questions:

- What is CSRF?
- Why does Laravel require a CSRF token?
- What happens if the token is missing?

Lab 4: Retrieve Old Input

Objective: Learn how to retrieve previously submitted form data.

Problem Statement:

Create a student registration form containing:

- Student name
- Email
- Phone
- Address

If the form submission fails validation, previously entered values should remain in the form.

Requirements:

- Use old('name').
- Use old('email').
- Use old('phone').
- Use old('address').
- Display old values after validation failure.

Expected Behaviour:

If a student enters their name and email but submits an invalid phone number, the name and email should remain populated after the redirect.

Lab 5: Display Validation Errors

Objective: Display validation errors using Blade.

Problem Statement:

Create a student registration form with the following validation:

Field	Validation
Name	Required
Email	Required, valid email
Phone	Required
Age	Required, integer

Requirements:

- Use $request->validate().
- Display all errors using $errors->all().
- Display individual errors using @error.
- Apply CSS styling to error messages.

Expected Output:

Validation errors

Please correct the following errors:

- The name field is required.
- The email field must be a valid email address.
- The phone field is required.

Example of validation feedback

Lab 6: Implement Flash Messages

Objective: Understand Laravel flash data.

Problem Statement:

Create an employee registration form. After successful submission, redirect the user to the registration page and display a success message.

Requirements:

- Use redirect().
- Use with() to store a flash message.
- Retrieve the message using session().
- Display the message in Blade.

Expected Output:

Employee registered successfully!

Lab 7: Implement Basic Validation Rules

Objective: Practise Laravel's built-in validation rules.

Problem Statement:

Create a user registration form with the following fields.

Field	**Validation rules**
Name	required, string, min:3, max:50
Email	required, email
Password	required, min:8
Age	required, integer, min:18
Phone	required, digits:10

Requirements:

- Implement all validation rules.
- Display custom error feedback for each field.
- Preserve old input after validation failure.

Lab 8: Implement Custom Validation Messages

Objective: Create user-friendly validation messages.

Problem Statement:

Create a college admission form with:

- Student name
- Email
- Date of birth
- Phone number
- Course

Implement validation with custom messages.

Example requirements:

```
$request->validate(
    [
        'name' => 'required|min:3',
        'email' => 'required|email',
        'phone' => 'required|digits:10',
    ],
    [
        'name.required' => 'Please enter your full name.',
        'name.min' => 'Name must contain at least 3 characters.',
        'email.required' => 'Email address is compulsory.',
        'email.email' => 'Please enter a valid email.',
        'phone.digits' => 'Phone number must contain exactly 10 digits.',
    ]
);
```

Task: Write the complete controller and Blade form.

Lab 9: Implement Unique Email Validation

Objective: Validate duplicate records using database validation rules.

Problem Statement:

Create a registration form where the email address must be unique in the students database table.

Requirements:

- Create a students table.
- Add name and email columns.
- Implement the unique validation rule.
- Prevent duplicate email registration.

- Display a custom error message.

Validation rule:

'email' => 'required|email|unique:students,email'

Lab 10: Implement Conditional Validation

Objective: Understand conditional validation rules.

Problem Statement:

Create a registration form with two account types:

- Personal
- Business

If the user selects Business, the Company Name field must become mandatory.

Requirements:

- Create an account type dropdown.
- Create a company name field.
- Use required_if.
- Display validation messages.

Validation example:

```
$request->validate([
    'account_type' => 'required|in:personal,business',
    'company_name' => 'required_if:account_type,business|nullable|string'
]);
```

Lab 11: Password Confirmation Validation

Objective: Understand password confirmation.

Problem Statement:

Create a password registration form containing:

- Password
- Confirm Password

Requirements:

- Password is required.
- Minimum password length is 8.
- Password confirmation must match.
- Display appropriate error messages.

Validation rule:

'password' => 'required|min:8|confirmed'

Remember that the confirmation input must be named password_confirmation.

Lab 12: Implement File Upload Validation

Objective: Validate uploaded files.

Problem Statement:

Create a student profile form that accepts:

- Student name
- Email
- Profile photograph

Requirements:

- Accept JPG, JPEG and PNG.
- Maximum file size: 2 MB.
- Validate the uploaded image.
- Display validation errors.
- Store the uploaded file using Laravel Storage.

Validation example:

```
$request->validate([
    'name' => 'required|string',
    'email' => 'required|email',
    'photo' => 'required|image|mimes:jpg,jpeg,png|max:2048',
]);
```